

COLECCIÓN RUST PÍXEL A PÍXEL

Cuaderno 2 de Rust

Estructura, Memoria y Colecciones en Rust

Recomendado a partir de los 16-17 años



Prototipo Editorial

Laboratorio de Código Instantáneo

Índice General

1. 📖 Introducción y Objetivos del Cuaderno	1
1.1. Objetivos del Cuaderno	1
2. 🏠 Divide y Vencerás: Funciones Avanzadas (fn)	3
2.1. Entradas y Salidas: Parámetros y valores de retorno (→)	3
2.2. La última línea sin punto y coma: Expresiones vs. Sentencias	4
2.3. Proyecto Intermedio: Modularizando la Calculadora Científica	5
2.3.1. Creación del proyecto: mi_calculadora	5
3. 🤝 Las Reglas de Convivencia: Introducción Intuitiva al Ownership	9
3.1. ¿Quién es el dueño del dato? La regla del propietario único	10
3.2. El peligro de la copia vs. el movimiento (Move)	10
3.3. Compartir es vivir: Referencias (&) y «Préstamos» (Borrowing)	12
3.4. Préstamos mutables (&mut): Solo puede quedar uno	13
3.5. Consumo (destrucción) de una variable cuando se pasa como argumento	14
4. 📦 Colecciones de Datos: Colecciones de datos: Arrays y Vectores	16
4.1. 💡 Semejanzas y Diferencias: ¿Quién es quién?	16
4.2. 📁 Arrays: Listas con tamaño grabado a fuego	16
4.2.1. Creación y métodos más importantes	16
4.2.2. Métodos y operaciones más importantes de un array	17
4.2.3. Propiedad (Ownership) en los Arrays	17
4.2.4. 💻 Ejemplo con Arrays	17
4.3. 📦 Vectores: La lista elástica	18
4.3.1. Creación y métodos más importantes	18
4.3.2. Propiedad (Ownership) en los Vectores	18
4.3.3. Ejemplo Completo con Vectores	18
4.3.4. Recorriendo el vector con bucles for	19
4.3.5. 🎮 Proyecto Intermedio: El Sistema de Puntuaciones de un Videojuego . . .	20
5. 🗺 El Mapa de la Memoria de tu Programa	26
6. El laberinto de las palabras: Strings a Fondo	29
6.1. El texto que no cambia (&str) vs. El texto dinámico (String)	29
6.2. Conversiones y trucos útiles: .to_string(), .push_str() y concatenación	30
7. 🖨 Proyecto Final: La Agenda de Contactos por Terminal	32
8. Ejercicios resueltos	42
8.1. Enunciados	42
8.1.1. Ejercicio 1: Expresiones vs. Sentencias	42
8.1.2. Ejercicio 2: El peligro del Movimiento (Move) con Strings	42
8.1.3. Ejercicio 3: Préstamos Mutables (&mut)	42
8.1.4. Ejercicio 4: Inicialización y Lectura de Arrays	42
8.1.5. Ejercicio 5: Trabajando con Vectores Dinámicos	42
8.1.6. Ejercicio 6: Casting de tipos	42
8.2. Soluciones	43

8.2.1. Ejercicio 1: Expresiones vs. Sentencias	43
8.2.2. Ejercicio 2: El peligro del Movimiento (Move) con Strings	43
8.2.3. Ejercicio 3: Préstamos Mutables (&mut)	44
8.2.4. Ejercicio 4: Inicialización y Lectura de Arrays	44
8.2.5. Ejercicio 5: Trabajando con Vectores Dinámicos	45
8.2.6. Ejercicio 6: Casting de tipos	45

1. Introducción y Objetivos del Cuaderno

¡Te damos la bienvenida al segundo volumen de **Cuadernos de Rust**! Si en el primer cuaderno diste tus primeros pasos escribiendo líneas de código secuenciales y entendiendo la sintaxis básica, ha llegado el momento de dar el salto hacia la madurez como programador en Rust.

En este cuaderno vamos a dejar atrás los programas sencillos de un solo bloque para aprender a diseñar software de verdad: organizado, eficiente y seguro. Para lograrlo, dividiremos nuestro código en funciones avanzadas y construiremos herramientas más complejas, como una calculadora científica modular.

Sin embargo, el verdadero plato fuerte de este volumen es el superpoder que hace único a Rust: el Ownership (Sistema de Propiedad). A través de analogías del mundo real y sin tecnicismos innecesarios, entenderás por qué Rust prescinde de un **recolector de basura** para ser rápido y no permite que cometas errores graves de memoria. Aprenderás las reglas de convivencia del código: quién es el «dueño» de un dato, cuándo se «mueve» y cómo se «presta» de forma segura.

Finalmente, descubriremos cómo almacenar grandes volúmenes de información utilizando **Arrays** y **Vectores**; entenderemos cómo se organiza la memoria de tu ordenador y desmitificaremos el trabajo con textos mediante el control total de los Strings. ¡Prepárate, porque al terminar este cuaderno habrás programado tu propia Agenda de Contactos funcional por terminal!

1.1. Objetivos del Cuaderno

Al finalizar este cuaderno, serás capaz de:

-  Dominar las funciones avanzadas: Diseñar código modular y limpio, comprendiendo la diferencia crítica en Rust entre una expresión y una sentencia (el misterio del punto y coma).
-  Pensar en términos de Ownership: Asimilar de forma intuitiva las reglas de propiedad, movimientos (moves) y préstamos (borrowing) mutables e inmutables para escribir código seguro por defecto.
-  Manipular colecciones de datos con soltura: Elegir con criterio cuándo utilizar un Array estático grabado a fuego o un Vector elástico que crece y se contrae dinámicamente según las necesidades de tu programa.
-  Visualizar la memoria: Comprender el mapa mental de qué pasa dentro de tu ordenador cuando ejecutas un programa y cómo Rust gestiona los recursos de forma automática.
-  Dominar el laberinto de los Strings: Trabajar sin miedo con textos estáticos (&str) y dinámicos (String), sabiendo cómo concatenarlos, convertirlos y manipularlos.

-  Construir aplicaciones reales: Integrar todos los conceptos del cuaderno para dar vida a proyectos prácticos, desde el sistema de puntuación de un videojuego hasta una base de datos local de contactos.

2. 🏠 Divide y Vencerás: Funciones Avanzadas (fn)

Hasta ahora, salvo en una breve alusión a las funciones en el Cuaderno 1, has escrito códigos donde las cosas pasaban una detrás de otra dentro de la función principal `main()`. Esto funciona para programas pequeños, pero si tu código crece, se vuelve un caos indescifrable.

Imagina que estás programando un videojuego: no metes el código de calcular la física (gravedad, rebotes, etc), pintar los gráficos y reproducir el sonido en una sola bolsa gigante. Lo separas en tareas especializadas. Esas tareas son las funciones.

2.1. Entradas y Salidas: Parámetros y valores de retorno (→)

Una función se comporta como una máquina de una fábrica: le introduces una materia prima (**parámetros**), realiza un proceso interno, y te devuelve un producto terminado (valor **de retorno**).

En Rust, para que el compilador te proteja de errores, estás obligado a declarar de qué tipo es cada parámetro que entra y qué tipo de objeto va a salir.

Obsérvalo en el siguiente ejemplo:

```
fn calcular_edad(anio_actual: i32, anio_nacimiento: i32) -> i32 {
    let edad = anio_actual - anio_nacimiento;
    return edad; // Enviamos el resultado de vuelta a quien nos llamó
}

fn main() {
    let mi_edad = calcular_edad(2026, 2010); // Llamamos a la máquina
    println!("Tienes {} años. ¡Estás en la flor de la vida!", mi_edad);
}
```

La función anterior recibe dos parámetros de tipo `i32`: el año actual y el año de nacimiento, calcula la edad y la devuelve también como `i32` mediante la sentencia `return`.

Veamos como se ejecuta el programa entero. Empieza en la primera línea del `main()`.

El tipo de los parámetros y del valor de retorno se especifica en la cabecera de la función:

```
fn calcular_edad(anio_actual: i32, anio_nacimiento: i32) -> i32
```

La llamada a la función en el `main`:

```
let mi_edad = calcular_edad(2026, 2010); // Llamamos a la máquina
```

En esta línea declaramos la variable `mi_edad` con `let` (inmutable) y le asignamos el valor de retorno de la función `calcular_edad`, pasándole al parámetro `anio_actual` de la función el

argumento 2026 y al parámetro **anio_nacimiento** el **argumento** 2010. Esta distinción entre parámetros cuando son variables que figuran en la definición de la función y argumentos cuando son valores que precisamente se pasan a los parámetros, la volveremos a comentar más adelante.

Después de la línea anterior, la ejecución del programa pasa a la primera línea de la función, que hace la resta $2026 - 2010 = 16$ y vuelve al `main()` asignando este valor a `mi_edad`.

A continuación el `main()` imprime la línea:

Tienes 16 años. ¡Estás en la flor de la vida!

Y el programa termina.

2.2. La última línea sin punto y coma: Expresiones vs. Sentencias

Aquí viene uno de los «secretos» más curiosos de Rust y que más descoloca al principio. Rust es un lenguaje basado en expresiones. Esto significa que casi todo lo que se escribe en una línea de código devuelve un valor. No obstante, podemos diferenciar entre sentencias y expresiones:

- Sentencia (Termina en `;`): Es una orden directa. «Haz esto». No devuelve nada.
 - Ejemplo: **`let velocidad = 10;`**
- Expresión (NO lleva `;` al final): Es un cálculo. «Esto vale tanto». Devuelve un valor automáticamente.
 - Ejemplo: **`5 + 3`**

La expresión anterior podemos utilizarla en una sentencia **`let resultado = 5 + 3;`** como se muestra a continuación:

```
fn main() {
    // El cálculo "5 + 3" es la expresión que devuelve el valor 8
    let resultado = 5 + 3;

    println!("El resultado de la expresión es: {}", resultado);
}
```

En Rust, si la última línea de una función no tiene punto y coma, actúa como un `return` automático para devolver el valor. Mira cómo podemos simplificar la función anterior al «estilo Rust profesional» sin escribir explícitamente `return` en la línea final que devuelve el resultado:

```
fn calcular_edad_pro(anio_actual: i32, anio_nacimiento: i32) -> i32 {
    // ¡Sin ";" al final! Rust entiende que éste es el resultado a
    // devolver.
    anio_actual - anio_nacimiento
}
```

💡 Consejo: Si por error le pones un punto y coma a esa última línea, el compilador se quejará diciendo que la función devuelve «nada» () en lugar de un `i32` tal como decimos en la cabecera de la función al definirla. ¡Prueba a quitar ese punto y coma y verás la magia!

📘 **Nota:** Cuando una función no devuelve nada, el compilador lo indica mostrando que devuelve (). El paréntesis vacío equivale a nada.

2.3. Proyecto Intermedio: Modularizando la Calculadora Científica

¡Enhorabuena por llegar hasta aquí! En los apartados anteriores de este Cuaderno 2, has aprendido cómo Rust gestiona la estructura de tu código mediante **funciones**. Ahora es el momento de unir todas las piezas creando una **Calculadora Científica Modular**.

Nuestra calculadora permitirá realizar operaciones básicas (suma, resta) y algunas funciones más avanzadas muy utilizadas en ciencia e ingeniería: el cálculo de potencias y el factorial de un número.

Veremos además el significado de dos nuevas palabras de Rust: **mod** (que significa modulo) y **pub** (que significa publico). Esto nos va a permitir estructurar el programa en módulos o ficheros independientes; concretamente el módulo principal **main.rs** que utilizará el módulo **operaciones.rs** el cual contiene las funciones.

La definición de las funciones dentro de **operaciones.rs** va precedida de la palabra **pub** lo que permite (al ser públicas) que se puedan llamar desde el archivo **main.rs** (u otros archivos) como si las hubiéramos escrito dentro del archivo **main.rs**.

Por defecto, es decir, sino calificamos las funciones con **pub**, Rust las define como privadas y entonces no pueden llamarse desde fuera del archivo en donde están escritas.

A su vez, para poder utilizar las **funciones** del modulo **operaciones.rs** desde el módulo **main.rs**, debemos incluir en este último archivo la instrucción **mod operaciones**; para que **main.rs** reconozca este archivo.

2.3.1. Creación del proyecto: `mi_calculadora`

En el Cuaderno 1 apartado 6 «🔧 Subiendo de nivel: Instalación de un Entorno de Desarrollo Profesional» se explica de forma detallada **cómo instalar Rust de forma profesional en tu ordenador y cómo crear un proyecto**.

Revisa esa parte del Cuaderno 1 si lo necesitas y crea el proyecto: **mi_calculadora**. Añade al directorio **src** el fichero **operaciones.rs**.

1. La Estructura del Proyecto

Para este proyecto, nuestro programa se dividirá en dos partes fundamentales:

- **main.rs**: El motor principal que interactúa con el usuario.
- **operaciones.rs**: El módulo (o caja de herramientas) que albergará las funciones matemáticas complejas.

Nuestra carpeta del proyecto en Cargo debe lucir exactamente así:

```

mi_calculadora/
├── Cargo.toml
└── src/
    ├── main.rs
    └── operaciones.rs

```

2. El Código del Proyecto

Archivo: **src/operaciones.rs**

Este archivo no tiene una función main. Es una «biblioteca» matemática. Fíjate bien en la palabra clave **pub** (público), imprescindible para que otros archivos puedan usar estas funciones:

📘 **Nota:** Si la última línea de una función NO TERMINA en punto y coma ; es una expresión y el resultado de evaluar dicha expresión es devuelto automáticamente por la función sin necesidad de utilizar return. El valor se devuelve a la variable que se utiliza cada vez que se llama la función.

```

// src/operaciones.rs

// Devuelve la suma de dos números flotantes
pub fn sumar(a: f64, b: f64) -> f64 {
    a + b // Expresión: devuelve el resultado directamente
}

/// Devuelve la resta de dos números flotantes
pub fn restar(a: f64, b: f64) -> f64 {
    a - b
}

/// Calcula la potencia de una base elevada a un exponente entero.
pub fn calcular_potencia(base: f64, exponente: i32) -> f64 {
    base.powi(exponente)
}

/// Calcula el factorial de un número entero de forma iterativa.
pub fn calcular_factorial(n: u64) -> u64 {
    let mut resultado = 1;
    for i in 1..=n {
        resultado *= i;
    }
    resultado
}

```

La expresión **for i in 1..=n** es un bucle que se repite **n** veces. La primera vez **i** vale 1, la segunda vez **i** vale 2 y la enésima vez **i** vale **n**.

❗ **Nota:** Este tipo de bucles está explicado en el Cuaderno 1, apartado '4.4 El bucle for con rangos'. Si quieres refrescar la memoria prueba simplemente a calcular_factorial(4) que debe darte $4 \times 3 \times 2 \times 1 = 24$

Archivo: src/main.rs

Aquí es donde el programa cobra vida. Usamos **mod operaciones**; para indicarle a Rust que incluya en el **main.rs** las herramientas que acabamos de crear en **operaciones.rs**.

```
// src/main.rs

// 1. Declaramos el módulo externo para que Rust sepa que existe
mod operaciones;

// 2. Traemos las funciones al entorno actual para usarlas
use operaciones::{sumar, restar, calcular_potencia,
calcular_factorial};

fn main() {
    println!("=== CALCULADORA CIENTÍFICA MODULAR ===");

    // Ejemplo 1: Probando la suma (f64 + f64)
    let a = 2.0;
    let b = 3.0;
    let suma = sumar(a, b);
    println!("La suma de {} y {} es {}", a, b, suma);

    // Ejemplo 2: Probando la resta (f64 - f64)
    let c = 20.0;
    let d = 5.0;
    let resta = restar(c, d);
    println!("La resta de {} menos {} es {}", c, d, resta);

    // Ejemplo 3: Probando la potencia (f64 elevado a i32)
    let base = 2.5;
    let exp = 3;
    let potencia = calcular_potencia(base, exp);
    println!("La potencia de {} elevado a {} es: {}", base, exp,
potencia);

    // Ejemplo 4: Probando el factorial (u64)
    let numero = 5;
    let factorial = calcular_factorial(numero);
    println!("El factorial de {}! es: {}", numero, factorial);
}
```

3. Explicación del Ejemplo (Paso a Paso)

Para entender por qué este código funciona tan bien, analicemos sus tres pilares clave:

- **El Sistema de Módulos (`mod` y `pub`):** Por defecto, todo en Rust es privado (está oculto). Si en `operaciones.rs` hubiéramos escrito `fn calcular_factorial` sin el **`pub`** delante, `main.rs` no podría ver la función y el compilador daría un error. Al añadir `pub`, le damos permiso a otros archivos para usarla.
- **El Método `powi`:** Rust es extremadamente estricto con los tipos de datos. En lugar de un método genérico para calcular potencias, usamos `.powi()`, que significa «*Power Integer*» (potencia entera). Permite elevar de forma súper eficiente un número con decimales (f64) a un exponente entero (i32).
- **Mutabilidad Controlada y Rangos Especiales:** En la función del factorial, declaramos **`let mut resultado = 1;`**. Como en Rust las variables son inmutables por defecto para evitar despistes, la palabra clave **`mut`** le avisa al compilador de que ese valor sí va a cambiar dentro del bucle. Además, el bucle usa el operador **`1..=n`** (rango inclusivo), lo que asegura que el propio número `n` se incluya en la multiplicación.

3. 🤝 Las Reglas de Convivencia: Introducción Intuitiva al Ownership

Llegamos al núcleo de lo que hace a Rust un lenguaje único en el mundo: el Ownership (Propiedad).

Para entenderlo, imagina que cuando tu programa quiere guardar el valor de una variable, primero debe pedirle al sistema operativo un «hueco» o reserva de memoria RAM donde quepa ese valor.

El problema es que, muchas veces, unas líneas más abajo en el código dejamos de usar esa variable, pero la reserva de memoria sigue activa. Si esto ocurre dentro de una función que se repite miles de veces, el programa irá acumulando memoria ocupada e inútil hasta que el ordenador se sature y falle. Este es el famoso *Memory Leak* (fuga de memoria), y es solo uno de los muchos dolores de cabeza que existen.

¿Cómo se soluciona esto? Liberando la memoria cuando ya no se usa. Cada lenguaje lo hace a su manera:

- **Java, Python o C#:** Tienen un Recolector de Basura (Garbage Collector). Es un programa vigilante en segundo plano que limpia la memoria que ya no se usa de forma automática, pero a costa de ralentizar el programa principal.
- **C o C++:** Todo depende de la destreza del programador, que debe escribir a mano cuándo liberar la memoria. Un solo olvido provoca fallos graves de seguridad o cierres inesperados.
- **Rust:** ¡Toma un camino revolucionario! No tiene un recolector que vuelva lento el programa, pero tampoco deja la responsabilidad en manos del programador. En Rust, **el propio compilador añade el código de limpieza de forma automática**, asegurándose de que la memoria se libere exactamente en el momento adecuado.

El compilador de Rust aplica las tres reglas que mencionamos a continuación para controlar las reservas y liberaciones de memoria:

Valor y dato tienen el mismo significado.

1. Cada valor o dato en Rust tiene un dueño (owner).
2. Solo puede haber un dueño a la vez.
3. Cuando el dueño sale del ámbito (scope), el valor se destruye.

En los tres puntos anteriores, un dueño es una variable que maneja el valor o dato en la memoria.

Para comprender el significado de ámbito (scope) veamos el siguiente ejemplo:

```
fn main() {
    let nombre = "Diego";
```

```

{
    let color = "rojo";
    println!("{color}");
} // <-- Final del ámbito de la variable color. "rojo" se destruye

println!("{nombre}");
} // <-- Final del ámbito de la variable nombre. "Diego" se destruye

```

Como vemos en el listado, el ámbito de una variable termina en la llave de cierre del bloque en la que está definida.

3.1. ¿Quién es el dueño del dato? La regla del propietario único

En Rust:

Cada dato en memoria tiene una variable que es su dueño (owner).

Solo puede haber un dueño a la vez.

En el listado del apartado anterior, el dueño del dato «Diego» es la **variable** nombre y el dueño del dato «rojo» es la **variable** color.

Cuando la variable queda fuera de ámbito porque el programa ha llegado hasta el final de su ámbito, el dato que representa la variable se destruye automáticamente para no ocupar espacio en la memoria del ordenador.

3.2. El peligro de la copia vs. el movimiento (Move)

En este apartado vamos a ver **como cambia el propietario de un dato** cuando hacemos una asignación de una variable a otra. Esto ocurre cuando se trata de datos complejos como textos dinámicos. Veremos como ejemplo el caso de un String (Texto).

Existen al menos dos formas de definir un string:

```

let mi_string1 = "Hola Mundo".to_string();
let mi_string2 = String::from("Hola Mundo");

```

Las dos variables definidas son de tipo **String** y representan la misma cadena de texto en ambos casos: **Hola Mundo**.

Veamos en el siguiente ejemplo como el dato, el String **Spiderman: Año Uno** cambia de dueño al hacer una asignación de variables.

Con la terminal abierta en tu carpeta de proyectos de Rust, haz clic derecho sobre la carpeta y **crea una terminal integrada de Visual Studio Code**. Crea un proyecto de Cargo con el comando **cargo new movimiento**, sustituye el código del **main.rs** por el siguiente listado y ejecuta el programa mediante **cargo run**.

 Proyecto: **movimiento**

```
fn main() {

    // El dueño del dato es comic_original
    let comic_original = String::from("Spiderman: Año Uno");

    // El dato SE MUEVE de dueño
    // El dueño del dato es ahora "otro_comic"
    let comic_prestado = comic_original;

    // Aquí la variable comic_original ya no existe

    //   --- ERROR COMPILADOR: Intentas leer algo que ya no
    existe
    // println!("Voy a releer mi cómic: {}", comic_original);

    println!("Mi amigo está leyendo: {}", comic_prestado); // Esto sí
    funciona
}
```

En el código anterior, al hacer la asignación **let comic_prestado = comic_original;**, el dueño del dato **Spiderman: Año Uno** se mueve de **comic_original** (que era su dueño original) a **comic_prestado** que es su nuevo y único dueño.

Además, la variable **comic_original** se destruye y deja de existir.

Hemos dicho que esto solo ocurre cuando se trata de datos complejos como textos dinámicos, y hemos puesto como ejemplo un String para que puedas comprobarlo.

Descomenta el último println! e intenta compilar.

Sin embargo, esto no ocurre cuando se trata de los tipos básicos mencionados en el Cuaderno 1. Veamos un ejemplo:

```
fn main() {
    let a = 10;
    let mut b = a;

    println!("{a}"); // 10
    println!("{b}"); // 10

    b = 16;
    println!("{a}"); // 10
    println!("{b}"); // 16
}
```

Como no hemos especificado el tipo de las variables a y b, el compilador las tomará por defecto como **i32**, un tipo básico.

La variable **b** la hemos hecho mutable **let mut b** para poder cambiar su valor después de su primera asignación y comprobar que evoluciona por si sola independiente de **a**.

La asignación **let mut b = a;** no consume en este caso la variable **a**, que no se destruye, ya que como podemos comprobar la utilizamos posteriormente imprimiéndola en la instrucción **println!("{a}");**.

Esto ocurre, como hemos mencionado, porque las variables **a** y **b** son de **tipo básico**, concretamente **i32**. Con los **String** ya vimos también que las asignaciones funcionan de manera diferente.

Los tipos básicos son todos los descritos en el apartado **5.1.4.1 Tipos escalares** del Cuaderno 1.

3.3. Compartir es vivir: Referencias (&) y «Préstamos» (Borrowing)

Como ir perdiendo la propiedad de tus variables cada vez que las asignas a otra variable es un dolor de cabeza, Rust inventó las Referencias (&) también llamadas préstamos. En lugar de regalar el cómic y perder la propiedad, lo dejas prestado para que lo lean.

El símbolo **&** delante de una variable significa: «Te dejo mirar este dato, te lo presto, pero sigue siendo mío».

En el siguiente ejemplo, la función **mostrar_info_comic** admite un parámetro denominado **comic** del tipo **&String**. Es por tanto un préstamo de un tipo **String**. El cuerpo de la función imprime el argumento que se le pase al parámetro **comic**.

 Copia el siguiente listado en la Playground y ejecuta el programa pulsando sobre el botón [RUN].

```
// Esta función solo "mira" el texto, para imprimirlo.
// No se adueña de él. Usa &String
fn mostrar_info_comic(comic: &String) {
    println!("Examinando el ejemplar: {}", comic);
}

fn main() {
    // Creamos el String (el dato)
    let mi_comic = String::from("Batman: El caballero oscuro");

    // Pasamos el cómic con un "&" delante. ¡Es un préstamo!
    mostrar_info_comic(&mi_comic);

    // ¡Buenas noticias! Como solo lo prestamos para mirar, seguimos
    // siendo los dueños.
    println!("Lo guardo en mi estantería: {}", mi_comic);
}
```

Cuando llamamos la función en el **main** con **mostrar_info_comic(&mi_comic);**, le pasamos un préstamo de **mi_comic** a la función para que lo pueda imprimir pero como no le hemos pasado la variable completa **mi_comic**, sino solamente un préstamo; la variable **mi_comic** sigue activa después de la llamada a la función; no se ha consumido y podemos imprimirla.

3.4. Préstamos mutables (&mut): Solo puede quedar uno

Ya hemos visto que con `&` podemos prestar un dato para que otros lo lean. Pero, ¿qué pasa si queremos prestar algo para que lo modifiquen? Imagina que le dejas tu cuaderno a un compañero para que corrija un ejercicio. Necesitas hacer un préstamo mutable, y en Rust esto se escribe con `&mut`.

Para evitar que el código se vuelva un caos y dos partes del programa intenten escribir a la vez sobre el mismo dato, Rust impone una regla de oro muy estricta: **mientras un dato esté prestado para modificarse (&mut), nadie más puede mirarlo ni modificarlo a la vez.**

Solo puede haber un préstamo mutable activo.

Mira este ejemplo de cómo funciona un taller de pintura de coches:

```
// Esta función recibe un coche modifcable usando &mut String
fn pintar_coche(coche: &mut String) {
    coche.push_str(" con alerón y pintura metalizada"); // Modificamos
    el texto original
}

fn main() {
    // 1. El coche original TIENE que ser mutable (mut) para poder
    cambiarlo
    let mut mi_coche = String::from("Seat Ibiza Blanco");

    // 2. Lo prestamos al taller usando &mut
    pintar_coche(&mut mi_coche);

    // 3. El coche original ha cambiado permanentemente en la memoria
    println!("Resultado del taller: {}", mi_coche);
    // Imprime: Resultado del taller: Seat Ibiza Blanco con alerón y
    pintura metalizada
}
```

 Ejecútalo en la Playground de Rust.

- La línea `coche.push_str(" con alerón y pintura metalizada");` añade al final del contenido de la variable `coche` —que tiene que ser de tipo `String`—, el texto " con alerón y pintura metalizada".
- En la línea `pintar_coche(&mut mi_coche);` le pasamos a la función `pintar_coche` el préstamo mutable `&mut mi_coche` para que la función, en su interior, pueda modificar la variable `mi_coche` como vemos al imprimirla inmediatamente después.

Como a la función le hemos pasado un préstamo, en este caso mutable, la variable `mi_coche` no se consume (no se destruye) en el préstamo a la función. La función modifica la variable y podemos imprimirla después, luego sigue activa.

 Un error muy común al empezar: Si intentas crear dos referencias mutables de la misma variable al mismo tiempo, el compilador te detendrá con un mensaje de error claro: **cannot borrow as mutable more than once at a time**. ¡Rust cuida que nadie rompa la memoria de tu ordenador!

Lo podemos comprobar escribiendo de una forma ligeramente distinta el programa anterior en el que definimos dos referencias mutables a `mi_coche`.

 Ejecuta el programa en la Playground para obtener el error.

```
// Esta función recibe un coche modifcable usando &mut String
fn pintar_coche(coche: &mut String) {
    coche.push_str(" con alerón y pintura metalizada"); // Modificamos
    el texto original
}

fn main() {
    // 1. El coche original TIENE que ser mutable (mut) para poder
    cambiarlo
    let mut mi_coche = String::from("Seat Ibiza Blanco");

    // 2. Definimos dos referencias mutables a mi_coche
    let referencia_mut_1 = &mut mi_coche;
    let referencia_mut_2 = &mut mi_coche;    // ✗ ERROR:

    // 2. Lo prestamos al taller usando referencia_mut_1
    pintar_coche(referencia_mut_1);

    // 3. El coche original ha cambiado permanentemente en la memoria
    println!("Resultado del taller: {}", mi_coche);
    // Imprime: Resultado del taller: Seat Ibiza Blanco con alerón y
    pintura metalizada
}
```

Comenta la línea `let referencia_mut_2 = &mut mi_coche;` con dos barras `//` al principio y vuelve a ejecutar el programa. El error ha desaparecido.

3.5. Consumo (destrucción) de una variable cuando se pasa como argumento

En los dos casos anteriores en los que hemos pasado referencias (péstamos) de una variable a una función, en el primer caso inmutable y en el segundo mutable, hemos comprobado que la variable original sigue estando activa y podemos imprimirla después de la llamada a la función.

Vamos a ver a continuación con un ejemplo que si pasamos la variable original (no una referencia) a la función, después de la llamada a la función la variable original se ha consumido (se ha destruido, ya no es accesible.)

 Ejecuta el siguiente programa en la Playground de Rust:

```
// Esta función recibe un coche modifcable de tipo String
// Lo indica en su parámetro: 'mut coche: String'
fn pintar_coche(mut coche: String) {
    coche.push_str(" con alerón y pintura metalizada"); //Modificamos el
```

```

texto original
}

fn main() {
    let mut mi_coche = String::from("Seat Ibiza Blanco");

    pintar_coche(mi_coche);
    println!("Resultado del taller: {}", mi_coche); // ✗ ERROR:

    println!("Final del programa")

}

```

Al pulsar el botón de [RUN] obtendrás un mensaje de error del compilador porque después de pasar **mi_coche** a la **función pintar_coche**, operación **en la cual se consume la variable mi_coche**, le pasamos luego otra vez la variable **mi_coche** que **ya no existe** a la función `println!`.

Comenta la línea con dos barras al principio:

```
// println!("Resultado del taller: {}", mi_coche);
```

Vuelve a ejecutar el programa y comprueba que funciona.

4. Colecciones de Datos: Colecciones de datos: Arrays y Vectores

Hasta ahora, nuestras variables eran como cajas individuales donde guardábamos un solo dato: un número, un texto o un booleano. Pero en la vida real, los programas manejan listas de cosas: las puntuaciones de una partida, los nombres de los alumnos de clase o los artículos de un inventario.

Para gestionar estas listas, Rust nos ofrece dos herramientas fundamentales: los **Arrays** y los **Vectores**.

4.1. Semejanzas y Diferencias: ¿Quién es quién?

Antes de verlos en código, hagamos una comparativa rápida para saber en qué se parecen y en qué se diferencian:

- Semejanzas: Ambos son colecciones **homogéneas**. Esto significa que todos los elementos de la lista tienen que ser obligatoriamente del mismo tipo de dato (todo números enteros, todo textos, etc.). No puedes mezclarlos. Además, en ambos casos el primer elemento de la lista está en la **posición 0**.
- Diferencias: La diferencia clave está **en el tamaño que ocupan durante el programa y en cómo se alojan en la memoria del ordenador**.
 - ▶ Un Array tiene un **tamaño fijo** que se decide al escribir el código y nunca puede cambiar. Se guarda en el Stack (una zona de la memoria ultra rápida).
 - ▶ Un Vector tiene un **tamaño dinámico**: puede crecer o encogerse mientras el programa se ejecuta. Se guarda en el Heap (una zona de memoria más flexible).

¿Para qué sirve cada uno?

- Usarás un **Array** cuando sepas de antemano el **número exacto de elementos** que va a tener y éste **nunca vaya a cambiar** (por ejemplo: los 7 días de la semana, las 4 estaciones o las coordenadas X e Y de un punto).
- Usarás un Vector cuando la **lista sea totalmente impredecible** (por ejemplo: los enemigos que aparecen en pantalla, los mensajes de un chat o la lista de la compra de un usuario).

4.2. Arrays: Listas con tamaño grabado a fuego

Estudiaremos en primer lugar los arrays porque son más sencillos de manejar que los vectores. Debido a que una vez creados no pueden cambiar de tamaño, el compilador los almacena en el Stack y Rust los maneja de una forma ultrarápida.

4.2.1. Creación y métodos más importantes

Para crear un array, encerramos los valores entre corchetes []. Rust nos permite declarar su tipo y su tamaño fijo con la sintaxis [tipo; tamaño]:

Veamos un par de ejemplos:

```
// Un array de 4 números flotantes f32
let temperaturas: [f32; 4] = [15.5, 22.1, 30.4, 18.2];

// Truco: Crear un array de 500 elementos todos cero
// Como no hemos indicado nada, por defecto serán i32
let contador_ceros = [0; 500];
```

4.2.2. Métodos y operaciones más importantes de un array

Supongamos que tenemos declarado el array **temperaturas**

```
let temperaturas: [f32; 4] = [15.5, 22.1, 30.4, 18.2];
```

- Acceso por índice: **temperaturas[0]** nos da el primer elemento (15.5).
- Conocer su tamaño: **temperaturas.len()** nos dice cuántos elementos tiene.
- Comprobar si está vacío: **temperaturas.is_empty()** devuelve true o false.

Veamos un sencillo ejemplo:

```
fn main() {
    let temperaturas: [f32; 4] = [15.5, 22.1, 30.4, 18.2];
    println!("Primera temperatura: {}", temperaturas[0]);
    println!("Longitud del array temperaturas: {}",
temperaturas.len());
    println!("¿Está vacío el array temperaturas?: {}",
temperaturas.is_empty());
}
```

 Ejecutando el programa en la Playground obtendrás el resultado:

```
Primera temperatura: 15.5
Longitud del array temperaturas: 4
¿Está vacío el array temperaturas?: false
```

4.2.3. Propiedad (Ownership) en los Arrays

En Rust, los tipos de datos simples (como números enteros, decimales y booleanos) tienen el rasgo Copy. Esto significa que cuando creas un array de números, los datos se copian fácilmente. Si pasas el array a otra variable, ambas variables seguirán funcionando de forma independiente. El array original no se destruye ni se «mueve» en la asignación.

Si los elementos del array son de tipo String el funcionamiento es distinto.

4.2.4. Ejemplo con Arrays

Imagina que queremos registrar las notas de los 3 trimestres de un alumno:

```
fn main() {
    // Creamos un array fijo de 3 elementos
    let notas_trimestre: [f32; 3] = [8.5, 9.0, 7.5];

    println!("Has cursado {} trimestres.", notas_trimestre.len());
    println!("La nota del segundo trimestre fue: {}",
notas_trimestre[1]);
}
```

Si intentas acceder a `notas_trimestre[4]` el compilador te lanzará un error porque sabe perfectamente que el array solo llega hasta el índice 2 (0, 1, 2). ¡Seguridad ante todo!

4.3. Vectores: La lista elástica

4.3.1. Creación y métodos más importantes

Los vectores se representan como **Vec** (donde la T significa el tipo de dato de los elementos que guardará). La forma más común e intuitiva de crearlos es usando el «atajo» o macro **vec!**:

```
// Un vector mutable con 2 textos dinámicos
let mut mochila = vec![String::from("Linterna"),
String::from("Cuerda)];
```

Como los vectores pueden cambiar de tamaño, tienen métodos específicos muy potentes:

- `.push(valor)`: Añade un elemento al final de la lista.
- `.pop()`: Quita el último elemento de la lista y te lo devuelve.
- `.insert(índice, elemento)`: Mete un elemento en la posición exacta (índice) que tú quieras, desplazando los demás.
- `.remove(índice)`: Borra el elemento de esa posición (índice).
- `[posicion]`: Te permite leer qué hay en una posición concreta. (Recuerda: ¡En informática siempre empezamos a contar desde la posición 0!)

4.3.2. Propiedad (Ownership) en los Vectores

Aquí es donde Rust se pone serio. Un vector es dueño de los datos que contiene. Si tu vector guarda elementos complejos (como `String`), no puedes simplemente sacar un elemento asignándolo a otra variable, porque estarías intentando «mover» la propiedad de una parte interna del vector, y Rust no lo permite.

Para leer datos de un vector sin romper el programa, casi siempre utilizaremos referencias (`&`) (préstamos para mirar).

4.3.3. Ejemplo Completo con Vectores

 Ejecuta el siguiente programa en la Playground

Vamos a simular el inventario de un jugador en un videojuego de rol (RPG):

```
fn main() {
    // 1. Inicializamos la mochila con dos ítems. Debe ser mutable
    (mut) para añadir cosas
    let mut inventario = vec![String::from("Poción de vida"),
String::from("Escudo de madera)];

    // 2. El jugador encuentra un objeto y lo guarda
    inventario.push(String::from("Espada de hierro"));

    // 3. Queremos mirar qué hay en la primera posición.
    // ¡Usamos "&" para pedirlo prestado! Si no ponemos "&", Rust dará
    error.
    let primer_objeto = &inventario[0];
    println!("El primer objeto de tu inventario es: {}",
primer_objeto);

    // 4. El jugador usa el último objeto que recogió (la espada)
    // .pop() saca el elemento del vector, reduciendo su tamaño a 2.
    inventario.pop();

    println!("Artículos restantes en la mochila: {:?}", inventario);
}
```

4.3.4. Recorriendo el vector con bucles for

Ya sabes guardar datos en un vector y cómo extraer elementos sueltos. Pero el verdadero potencial de las colecciones aparece cuando quieres procesar toda la lista de golpe: aplicar un filtro, calcular una media o imprimir un listado de alumnos.

Para hacer esto de forma automática, sin escribir código fila por fila, usamos el bucle **for** combinado con un préstamo (&). Al poner el & antes del vector, le estamos diciendo a Rust: «Déjame mirar uno a uno los elementos de la lista, pero no destruyas el vector al terminar».

Mira qué limpio queda el código para pasar lista en clase.

 Ejecuta el siguiente programa en la Playground

```
fn main() {
    let alumnos = vec![
        String::from("Lucía"),
        String::from("Marcos"),
        String::from("Sofía")
    ];

    println!("--- CONTROL DE ASISTENCIA ---");

    // "alumno" es una variable temporal que tomará el valor de cada
    elemento en cada vuelta
    for alumno in &alumnos {
        println!("👤 Presente: {}", alumno);
    }
}
```

```
// Como usamos "&alumnos", la lista sigue existiendo aquí
perfectamente
println!("Total de alumnos registrados: {}", alumnos.len());
}
```

💡 ¿Y si queremos modificar los elementos del vector mientras lo recorremos?

¡Fácil! Usamos un préstamo mutable tanto en el bucle como en el vector (**&mut**). Por ejemplo, si quisiéramos sumarle un punto de bonificación a una lista de notas por buen comportamiento:

🖥 Ejecuta el siguiente programa en la Playground

```
fn main() {
    let mut notas = vec![4.5, 6.0, 8.5];

    // Usamos &mut para tener permiso de escritura en cada nota
    // Ahora nota es una referencia por haber utilizado &mut
    for nota in &mut notas {

        // El asterisco (*) sirve para entrar "dentro" de la referencia
        // y cambiar el valor
        *nota += 1.0;
    }

    // Imprime: [5.5, 7.0, 9.5]
    println!("Notas con el punto extra: {:?}", notas);
}
```

No hemos estudiado todavía el concepto de referencia que va muy unido al de préstamo. En las estructuras for que hemos visto antes: **for nota in ¬as {...}** y **for nota in &mut notas {...}**, por haber utilizado el operador **&** de préstamo y **&mut** de préstamo mutable delante del array **notas**, la variable **nota** del for en cada iteración no contiene el valor de la nota sino la dirección de memoria (referencia) en donde se almacena la nota. Entonces, para llegar al contenido de la nota tenemos que utilizar el operador asterisco ***nota**.

4.3.5. 🎮 Proyecto Intermedio: El Sistema de Puntuaciones de un Videojuego

Vamos a construir el motor de puntuaciones para un juego arcade. Nuestra misión es diseñar un programa que registre las puntuaciones de las partidas, calcule estadísticas y determine si un jugador ha superado el récord histórico.

Este proyecto te servirá para consolidar el uso de vectores, bucles, condicionales y funciones con referencias (préstamos).

🔧 El Reto Práctico

Debes implementar un código en Rust que cumpla con los siguientes requisitos:

- Almacenar una lista dinámica con los puntos de las últimas partidas.

- Añadir nuevas puntuaciones a la lista simulando que se acaban de jugar esas partidas.
- Crear una función que calcule la puntuación media obtenida por el jugador.
- Crear una función que busque la puntuación más alta (el récord de la máquina).

⚙️ Explicaciones sobre el listado

Al final de este apartado te daremos el código completo del programa para que puedas crearte un proyecto con Cargo y ejecutarlo.

Antes de profundizar en el listado completo del programa, analicemos algunas funciones y fragmentos interesantes del mismo que nos ayudarán a comprender el programa completo. Sigue las explicaciones función a función que te damos aquí, pero no pierdas de vista su ubicación y funcionalidad en el programa completo que como hemos dicho se encuentra al final.

- Veamos la función **calcular_media**:

```
// Función que recibe un préstamo & del vector 'puntuaciones' de solo
// lectura
// y devuelve la media (f32)
fn calcular_media(puntuaciones: &Vec<i32>) -> f32 {
    if puntuaciones.is_empty() {
        return 0.0; // Si no hay partidas, la media es cero
    }

    let mut suma_total = 0;
    for puntos in puntuaciones {
        suma_total += *puntos; // Sumamos los puntos de cada partida a
        suma_total
    }

    // Convertimos a f32 para poder calcular decimales en la división
    suma_total as f32 / puntuaciones.len() as f32
}
```

Esta función recibe el argumento **puntuaciones** cuyo tipo debe ser **&Vec**, es decir, un préstamo inmutable de vector de elementos enteros. Al recibir un préstamo inmutable **&** solo podrá leer los elementos del vector pero no modificarlos. No obstante, para calcular la media no necesita nada más. La función devuelve un **f32**.

La primera consecuencia que queremos aclarar está precisamente en el bucle **for**. Fijémonos en la siguiente línea: **suma_total += *puntos;** Abajo explicamos el operador **+=** que es sencillo pero veamos el significado del asterisco delante de **puntos**. Cuando hacemos **for puntos in puntuaciones**, al ser **puntuaciones** un vector prestado (**puntuaciones: &Vec**), el **for** no nos devuelve en **puntos** el valor de la variable en cada iteración sino una referencia a la variable. Es decir, nos devuelve la dirección de memoria en donde está almacenada la variable **puntos**. Para obtener el valor de la variable **puntos**, tenemos que utilizar el operador de desreferencia, asterisco y, escribir ***puntos**.

Empecemos ahora por el inicio de la función. Si el vector **puntuaciones** no tiene ningún valor, entonces **puntuaciones.is_empty()** devuelve true, el primer if se cumple y la **función termina** con el **return** devolviendo 0.0 (un f32)

El siguiente bloque de la función calcula **la suma de todos los elementos del vector puntuaciones**; ya veremos cuando llamemos la función qué argumento le pasamos al parámetro puntuaciones.

Para entender este paso, aclaremos primero que la instrucción **suma_total += *puntos**; es equivalente a esta otra **suma_total = suma_total + *puntos**;

Por ejemplo, si nos situamos en la segunda vez (paso 2) que se entra en el bucle for.

La instrucción anterior es equivalente a:

suma_total(paso2) = suma_total(paso1) + *puntos(paso2)

Veamos ahora **cómo funciona el bucle for**:

```
let mut suma_total = 0;
for puntos in puntuaciones {
    suma_total += *puntos; // Sumamos los puntos de cada partida
}
```

- Supongamos que el vector **puntuaciones** tiene tres valores: 200, 100 y 300.
- Antes de entrar al bucle for le hemos dado a la variable **suma_total** el valor 0.
- la primera vez que la ejecución del programa entre en el for, ***puntos** valdrá 200 y entonces:

$\text{suma_total} = 0 + 200 = 200$

- la segunda vez que entre en el for, ***puntos** valdrá 100 y entonces:

$\text{suma_total} = 200 + 100 = 300$

- la tercera vez que entre en el for, ***puntos** valdrá 300 y entonces:

$\text{suma_total} = 300 + 300 = 600$, que representa la suma de los tres elementos.

Después del bucle for, la función calcula la **media de las puntuaciones** asignándole al resultado un tipo **f32**, que es lo que tiene que devolver la función. El código es este:

```
// Convertimos a f32 para poder calcular decimales en la división
suma_total as f32 / puntuaciones.len() as f32
```

Como **suma_total** es un entero y **puntuaciones.len()** el número de elementos que tiene el vector es otro entero, si los dividimos tal cual obtendremos otro entero, es decir, se desprecian los decimales al hacer la división. Sin embargo, hemos dicho en la cabecera de la función que tenemos que devolver un **f32**

Las palabras **as f32** permiten convertir en ambos casos las variables al tipo f32 y así efectuamos la división y obtenemos un f32 que es lo que tenemos que devolver.

Finalmente, como la última línea no termina en punto y coma, el resultado de evaluarla que no es más que la media de las puntuaciones es lo que devuelve la función.

- La función **obtener_record**:

Esta función se puede entender con las explicaciones dadas en la función **calcular_media** y las anotaciones en el cuerpo de la misma como vemos a continuación:

```
// Función que busca el récord del jugador
fn obtener_record(puntuaciones: &Vec<i32>) -> i32 {
    if puntuaciones.is_empty() {
        return 0;
    }

    // Empezamos asumiendo que la primera puntuación es la mayor
    // Si no es así luego la iremos cambiando
    let mut maximo = puntuaciones[0];

    // Si encontramos algún valor en puntuaciones que sea mayor
    // cambiaremos el valor anterior
    for puntos in puntuaciones {
        // El asterisco en *puntos es necesario para leer el valor de
        puntos
        // porque el vector puntuaciones es un préstamo & (igual que en
        la función anterior)
        if *puntos > maximo {
            // Si encontramos una mayor, actualizamos el récord
            maximo = *puntos;
        }
    }
    // Devolvemos el máximo encontrado
    maximo
}
```

- La función **main**

Aquí es donde realizamos, entre otras cosas, las llamadas a las funciones que hemos explicado antes. La vamos a explicar también sobre el código comentado:

```
fn main() {
    // Creamos la tabla de puntuaciones vacía del jugador
    // Vec::new() es un vector que no tiene elementos todavía
    let mut mis_partidas: Vec<i32> = Vec::new();

    // Simulamos que el jugador termina 3 partidas en la máquina
    arcade.
    // La instrucción: mis_partidas.push(2500); introduce el valor 2500
    // en el vector mis_partidas.
    // Las siguientes instrucciones .push(valor) añaden elementos al
    vector.
    mis_partidas.push(2500);
    mis_partidas.push(4200);
    mis_partidas.push(1800);
}
```

```

// Imprimimos el estado actual
println!("🎮 Puntuaciones de la sesión: {:?}", mis_partidas);

// Calculamos estadísticas llamando a nuestras funciones
especializadas

// Ejecutamos la función calcular_media y le pasamos como argumento
// &mis_partidas, es decir, un préstamo del vector mis_partidas.
Por tanto
// la función calculará y devolverá la media de los valores que
hemos
// introducido antes en mis_partidas.
// Lo que devuelve la función se coloca en la variable media
let media = calcular_media(&mis_partidas);

// La función obtener_record
// calcula y devuelve el record de las puntuaciones
let record = obtener_record(&mis_partidas);

println!("📊 Estadísticas del Jugador:");
println!("    -> Puntuación Media: {:.2} puntos", media);
println!("    -> Récord Actual: 🔥 {} puntos", record);

// Simulamos una última partida espectacular
let nueva_partida = 5000;
println!("\n🚀 ¡Nueva partida terminada! Consigues {} puntos.",
nueva_partida);

if nueva_partida > record {
    println!("🎉 ¡BRUTAL! Has batido tu propio récord histórico.");
}

mis_partidas.push(nueva_partida);
}

```

Listado completo del programa

El Sistema de Puntuaciones de un Videojuego

🖥️ Abre una terminal integrada en el directorio **proyectos-rust** que ya creaste en el Cuaderno 1 y crea un proyecto nuevo con la orden:

```
cargo new puntuaciones_videojuego
```

Copia el código del listado que te damos a continuación en el fichero **src/main.rs**.

Abre una terminal integrada en la carpeta **puntuaciones_videojuego** (el proyecto que acabas de crear) y ejecuta el programa mediante la orden **cargo run**.

```

// Función que recibe un préstamo del vector (solo lectura)
// de elementos enteros de 32 bits y devuelve la media (f32)
// que es un valor decimal de 32 bits
fn calcular_media(puntuaciones: &Vec<i32>) -> f32 {
    if puntuaciones.is_empty() {
        return 0.0; // Si no hay partidas, la media es cero
    }

    let mut suma_total = 0;
    for puntos in puntuaciones {
        suma_total += *puntos; // Sumamos los puntos de cada partida
    }

    // Convertimos a f32 para poder calcular decimales en la división
    suma_total as f32 / puntuaciones.len() as f32
}

// Función que busca el récord del jugador
fn obtener_record(puntuaciones: &Vec<i32>) -> i32 {
    if puntuaciones.is_empty() {
        return 0;
    }

    // Empezamos asumiendo que la primera es la mayor
    let mut maximo = puntuaciones[0];

    // Si encontramos algún valor en puntuaciones que sea mayor
    // cambiaremos el valor anterior
    for puntos in puntuaciones {
        // El asterisco en *puntos es necesario para leer el valor de
        puntos
        // porque el vector puntuaciones es un préstamo &
        if *puntos > maximo {
            // Si encontramos una mayor, actualizamos el récord
            maximo = *puntos;
        }
    }
    // Devolvemos el máximo encontrado
    maximo
}

fn main() {
    // Creamos la tabla de puntuaciones vacía del jugador
    let mut mis_partidas: Vec<i32> = Vec::new();

    // Simulamos que el jugador termina 3 partidas en la máquina arcade
    mis_partidas.push(2500);
    mis_partidas.push(4200);
    mis_partidas.push(1800);

    // Imprimimos el estado actual

```

```
println!("🎮 Puntuaciones de la sesión: {:?}", mis_partidas);

// Calculamos estadísticas llamando a nuestras funciones
especializadas
let media = calcular_media(&mis_partidas);
let record = obtener_record(&mis_partidas);

println!("📊 Estadísticas del Jugador:");
println!("    -> Puntuación Media: {:.2} puntos", media);
println!("    -> Récord Actual: 🔥 {} puntos", record);

// Simulamos una última partida espectacular
let nueva_partida = 5000;
println!("\n🚀 ¡Nueva partida terminada! Consigues {} puntos.",
nueva_partida);

if nueva_partida > record {
    println!("🎉 ¡BRUTAL! Has batido tu propio récord histórico.");
}

mis_partidas.push(nueva_partida);
}
```

5. 🗺 El Mapa de la Memoria de tu Programa

En el Cuaderno 1 **Descubriendo la Programación y Filtros Digitales** diferenciamos entre el **código fuente** (archivo que nosotros escribimos y es entendible por un humano) y el **código ejecutable** compuesto por unos y ceros, fruto del proceso de compilación y que es el que entiende el ordenador.

Como casi todos conocemos Windows podemos decir que cuando arrancamos un programa haciendo doble clic sobre un icono del escritorio, estamos cargando en la memoria RAM del ordenador el ejecutable de ese programa. ¿Qué almacenamos en la RAM?

Imagina la memoria RAM asignada a tu programa como un gran edificio de cuatro plantas:

1. 📖 El Segmento de Código (Text Segment)

Aquí es donde se guarda, literalmente, tu programa traducido a lenguaje de máquina (los ceros y unos que entiende el procesador). Es de sólo lectura. El ordenador lee las instrucciones de una en una para saber qué hacer, pero tu programa no puede modificarse a sí mismo mientras corre. ¡Sería peligrosísimo!

2. 🧑 El Segmento de Datos (Data Segment y BSS)

En esta zona se guardan las cosas que nacen con el programa y mueren con él. Por ejemplo, las variables globales o las constantes que declaras arriba del todo de tu código.

Todavía no las hemos utilizado pero puedes imaginarlas como variables que nacen en el momento que se carga el ejecutable en memoria y están disponibles hasta que el programa termina. Permanecen inmutables en el mismo sitio desde el segundo uno hasta que cierras la aplicación.

3. 🍷 El Stack (La Pila)

Imagina una pila de platos en un buffet de hotel. Cuando pones un plato nuevo, lo pones arriba del todo. Cuando coges un plato, coges también el de arriba. A esto los informáticos lo llaman estructura LIFO (Last In, First Out: el último que entra es el primero que sale).

En el **Stack** se guardan las variables locales de tus funciones (los números enteros, booleanos, etc.). Es un área ultraordenada, limpia y muy rápida. No pueden guardarse en el Stack aquellas variables cuyo tamaño es desconocido cuando se compila el programa.

4. 🌳 El Heap (El Montón)

Imagina un gran almacén caótico. Cuando necesitas espacio para algo grande o que no sabes cuánto va a medir, le gritas al encargado del almacén: «¡Oye, necesito sitio para un texto largo!». El encargado busca un hueco libre que sirva, mete el dato ahí y te da una ficha con la dirección exacta (un puntero) para que sepas dónde encontrarlo. Es un área enorme y flexible, pero más lenta de gestionar.

Los **vectores**, que pueden aumentar y disminuir el número de elementos mientras se ejecuta el programa, se almacenan en el **Heap**.

Los **String** se almacenan en el Heap pero los **&str** se almacenan en el Segmento de Código, apartado 1.

Cuando tenemos variables de tipos compuestos, por ejemplo un struct como vimos en el Cuaderno 1, generalmente los campos que son de tipos básicos (i32, 264, f32, bool,...) se almacenan en el stack y si tienen, por ejemplo, campos de tipo String, estos se almacenan en el Heap. No obstante, el compilador es siempre libre de hacer optimizaciones y lo que hemos dicho es solo de carácter general.

🔪 El Gran Duelo: ¿Stack o Heap?

Para entender por qué Rust es tan especial, necesitas comprender la diferencia radical entre estas dos áreas. Vamos a ponerlas cara a cara:

Característica	🍷 El Stack (La Pila)	🌳 El Heap (El Montón)
Tamaño del dato	Debe ser conocido y fijo antes de compilar.	Puede ser dinámico y cambiar sobre la marcha.
Organización	Perfecta y secuencial (un plato encima de otro).	Desparramada por el espacio libre disponible.
Velocidad	Velocidad de vértigo. El procesador sabe exactamente dónde está todo.	Más lento. Hay que buscar sitio libre y seguir «pistas».

Característica	 El Stack (La Pila)	 El Heap (El Montón)
¿Cuándo se limpia?	Automáticamente cuando la función termina (el plato se retira).	Hay que liberar el espacio a mano (o dejar que Rust lo haga por ti).

⚡ Tiempo de Compilación vs. Tiempo de Ejecución: El Secreto del Tamaño

Aquí está la clave de todo. Tu ordenador necesita saber cuánta memoria reservar para cada variable.

- Datos de **tamaño conocido** (Van al Stack): Si escribes **let edad: i32 = 16;**, el compilador sabe, antes de que nadie ejecute el programa, que un i32 ocupa exactamente 32 bits (4 bytes). Como el tamaño está grabado a fuego, Rust lo mete en el Stack. Es seguro, rápido y directo.
- Datos de **tamaño dinámico** (Van al Heap): Imagina que creas un programa para que el usuario escriba su nombre por teclado. ¿Mide igual lo que escribe si se llama «Ana» que si se llama «Alejandro»? ¡No! Como el tamaño no se conoce en tiempo de compilación (se conocerá cuando se ejecute el programa y el usuario entre el dato), Rust no puede meterlo en el Stack. En su lugar, pide espacio en el Heap en tiempo de ejecución, guardando allí el texto dinámico (String), mientras que en el Stack solo se queda una pequeña ficha de propiedad indicando dónde está ese montón de texto.

♂ El misterio de los &str y la Memoria de Sólo Lectura.

¿Te acuerdas de que en el tema anterior dijimos que los &str eran como carteles tallados en piedra? ¡Ahora vas a entender por qué de verdad!

Cuando escribes en tu código algo como:

```
let saludo = "Hola, chicos";
```

Ese texto «Hola, chicos» se incrusta directamente dentro del Segmento de Código (nuestra planta 1 del edificio, la de sólo lectura) en el momento de compilar.

La variable saludo no contiene el texto en sí. Es simplemente una referencia (&); una flecha mágica que apunta a esa zona de solo lectura. Por eso jamás puedes modificar un &str: ¡porque intentarías escribir en la zona prohibida de la memoria! Si necesitas cambiar el texto, estás obligado a usar un String para que el ordenador cree una copia moldeable en el Heap.

🌟 Conclusión: ¿Por qué os hacemos aprender esto si parece «magia»?

Admitámoslo: hoy en día los ordenadores son tan potentes que devoran gigabytes de memoria sin pestañear. Podríamos dejar que el compilador maneje la memoria como le dé la gana y olvidarnos de este rollo. Sin embargo, entender esto es vital por una gran razón: El **Ownership de Rust** (su sistema de propiedad) se diseñó exclusivamente para gestionar el Stack y el Heap sin fallos.

Cuando entiendes que:

- Una variable en el Stack es dueña de un espacio en el Heap.
- Si esa variable desaparece (sale de su función), el plato del Stack se quita.
- Al quitarse el plato, el espacio asignado en el Heap se limpia al instante...

... en ese preciso momento, los errores del compilador de Rust dejan de parecer un castigo y se convierten en tu mapa del tesoro. Dejas de pelear contra la «magia» y empiezas a diseñar software rápido, seguro y profesional.

6. El laberinto de las palabras: Strings a Fondo

En este tema hablaremos de los **strings** que representan **texto** y distinguiremos entre strings de tipo **&str**, que una vez creados no pueden cambiar y, strings de tipo **String**, que pueden ser modificados después de ser creados.

Si vienes de programar en lenguajes como Python, JavaScript o Java, las palabras y los textos eran simples: creabas una variable **texto** = "Hola" y te olvidabas de problemas. Pero en Rust, cuando intentas hacer operaciones sencillas con palabras, de repente el compilador te empieza a hablar de dos cosas llamadas **String** y **&str**.

¿Por qué Rust nos complica la vida con dos tipos de texto diferentes? La respuesta es, una vez más, el rendimiento y la seguridad de la memoria. Rust quiere que tu programa sea tan rápido como un videojuego de última generación, y para conseguirlo necesita tratar los textos fijos de una manera y los textos dinámicos (variables) de otra.

6.1. El texto que no cambia (&str) vs. El texto dinámico (String)

Para entender la diferencia de un vistazo, vamos a usar una metáfora:

&str (también llamado String Slice o Texto prestado): Imagina un letrero tallado en piedra. El **&str** es un texto estático, fijo, que se graba directamente dentro del propio archivo ejecutable de tu programa. Como está «tallado en piedra», ocupa un espacio fijo, no puede crecer, ni encogerse, ni modificarse mientras el programa se ejecuta. Es increíblemente rápido de leer.

String (También llamado Texto dinámico o En propiedad): Imagina un bloc de notas de anillas. **String** es una estructura de datos que se guarda en la memoria dinámica (Heap) de tu ordenador. Puedes arrancar páginas, escribir palabras nuevas al final, borrar letras o cambiar su tamaño en cualquier momento. Al ser dinámico, requiere un poquito más de trabajo para el ordenador, pero te da total libertad para su uso.

Mira la diferencia en el código:

```
fn main() {
    // 1. Esto es un &str. Un texto fijo "tallado" en el código.
    let mensaje_fijo: &str = "¡Bienvenido al nivel 2!";

    // Si intentas hacer:
```

```
// mensaje_fijo.push_str(" Crack"); -> ¡ERROR! No se puede alterar la
piedra.

// 2. Esto es un String mutable (mut).
// Un bloc de notas dinámico en el que podemos escribir.
let mut nombre_usuario: String = String::from("Gamer");

// ¡Aquí sí tenemos libertad! Podemos modificarlo y añadirle "Pro_99"
nombre_usuario.push_str("Pro_99");

println!("Saludo: {}, {}", mensaje_fijo, nombre_usuario);
// Imprime: ¡Bienvenido al nivel 2!, GamerPro_99
}
```

6.2. Conversiones y trucos útiles: .to_string(), .push_str() y concatenación

En el día a día con Rust, te vas a encontrar constantemente con que una función te pide un **String** pero tú tienes un **&str**, o viceversa. Aprender a pasar de un formato a otro es como aprender los cambios de marcha de un coche.

Aquí tienes el «manual de supervivencia» para trabajar con textos:

- **A) Pasar de Letrero de Piedra (&str) a Bloc de Notas (String)**

Si tienes un texto fijo y necesitas transformarlo en un texto mutable que pueda cambiar, usas **.to_string()** o **String::from()**:

```
fn main() {
    let texto_piedra: &str = "Hola";
    let texto_dinamico: String = texto_piedra.to_string(); //
¡Convertido!
    // o bien:
    let texto_dinamico2: String = String::from(texto_piedra);

    println!("{texto_dinamico}"); // Hola
    println!("{texto_dinamico}"); // Hola
}
```

Evidentemente, en el caso anterior `texto_dinamico` y `texto_dinamico2` son iguales. En ambos casos el compilador puede inferir el tipo de la variable y podemos escribir:

```
let texto_dinamico = texto_piedra.to_string(); // ¡Convertido!
let texto_dinamico2 = String::from(texto_piedra);
```

- **B) Pasar de Bloc de Notas (String) a Letrero de Piedra (&str)**

¡Esto es gratis! Como un **String** es el dueño de todo el texto, si solo quieres prestar una parte o su totalidad para que alguien lo lea como un **&str**, basta con ponerle el símbolo de préstamo (&) delante:

```
let bloc_notas: String = String::from("Contenido importante");

// Rust lo convierte automáticamente a &str
let prestamo_lectura: &str = &bloc_notas;
```

• C) Modificar un String sobre la marcha

Para añadir letras a un bloc de notas mutable, tienes dos herramientas clave:

1. **push()**: Añade un único carácter (va entre comillas simples, ej: "a").
2. **push_str()**: Añade una frase o palabra completa (va entre comillas dobles, ej: «hola»).

Ejemplo:

```
fn main() {
    // Para que un String sea mutable tenemos que declararlo con mut
    let mut palabra = String::from("Ruste");
    palabra.pop();           // Quita la 'e' final -> "Rust"
    palabra.push('a');       // Añade un carácter -> "Rusta"
    palabra.push_str("ceos"); // Añade un string slice (&str) ->
    "Rustaceos"

    println!("{}", palabra); // Rustaceos
}
```

• D) El truco de la macro format!

Intentar sumar varios textos en Rust usando el símbolo + puede volverse muy caótico por culpa de las reglas de propiedad (Ownership). La forma más elegante, limpia y profesional de combinar textos y variables en un nuevo String es usar la macro **format!**. Funciona exactamente igual que println!, pero en lugar de lanzar el texto por el altavoz de la pantalla, lo empaqueta y lo guarda en una variable.

```
fn main() {
    let clase = "4º ESO";
    let alumnos = 28;

    // format! une todo de forma segura y nos devuelve un String perfecto
    let informe: String = format!("Grupo: {} | Total alumnos: {}", clase,
    alumnos);

    println!("{}", informe); // Grupo: 4º ESO | Total alumnos: 28
}
```

7. Proyecto Final: La Agenda de Contactos por Terminal

¡Ha llegado el momento de la verdad! Vamos a construir una aplicación real desde cero: una **Agenda de Contactos interactiva** que se ejecuta en la terminal de tu ordenador. El programa mostrará **un menú en bucle** que permitirá al usuario **añadir** nuevos amigos, **listar** los contactos guardados y **buscar** un teléfono rápidamente por el nombre.

Resumiendo: vamos a hacer un programa que controle el Sistema de Puntuaciones de un cierto Videojuego.

El programa se basa en determinadas funciones que reciben como argumento un **préstamo de un vector**, en algunas ocasiones *immutable &* y en otras *mutable &mut*. Cuando el argumento que recibe la función es un préstamo immutable & solo queremos leer el valor de los elementos del vector que le pasamos a la función pero, si es mutable &mut, queremos modificar alguno o todos los elementos del vector.

En este proyecto del Cuaderno 2, juntaremos las piezas del puzle: Funciones, Vectores, Estructuras de control y manipulación de Strings.

El Diseño de la Aplicación

Para que el código esté limpio y ordenado (como el de los programadores profesionales), organizaremos la información utilizando dos vectores:

- **lista_nombres**: Un vector de tipo `Vec<String>` para almacenar los **nombres de pila**. La notación `Vec<String>` indica que todos los elementos del vector son de tipo `String`. Podemos definir, si queremos, vectores de cualquier tipo. Por ejemplo `Vec<i32>` sería un vector cuyos elementos son enteros de 32 bits.
- **lista_telefonos**: Un vector de tipo `Vec<String>` para guardar sus números de teléfono. En este caso los números se guardarán como si fueran texto, es decir, no se podrá operar matemáticamente con ellos.

Nota: La posición 0 de ambos vectores corresponderá al primer contacto, la posición 1 al segundo, y así sucesivamente.

Antes de empezar a analizar el programa en su totalidad, vamos a comentar la parte del mismo que se encarga de pedir datos al usuario.

Para ello, vamos a analizar en primer lugar un código reducido que pide al usuario que escriba algún dato y después de escribir y pulsar `return`, el programa devuelve lo que ha escrito el usuario incrustándolo en una frase. Este código reducido lo encontraremos prácticamente igual en el programa final.

 Abre un terminal integrado de Visual Studio Code en tu carpeta de proyectos-rust y crea un nuevo proyecto con la orden:

```
cargo new agenda_contactos
```

Se creará el directorio **agenda_contactos** y dentro el fichero `main.rs` en el interior de la carpeta **src**.

Copia el siguiente programa, que es solo una demo de cómo un programa solicita datos al usuario y los muestra y, pégalo en el fichero `main.rs` del proyecto que acabas de crear.

Ejecútalo, desde una terminal integrada abierta en la carpeta del proyecto, con **cargo run** y cuando te pida que introduzcas una palabra, tecléala y termina pulsando **Return**. Observa la salida por la pantalla.

Proyecto: **agenda_contactos**

```
use std::io::{self, Write};

fn main() {
    // Pedimos al usuario que introduzca una palabra
    print!("Introduce cualquier palabra: ");

    // Fuerza a la terminal a mostrar el texto en pantalla
    io::stdout().flush().unwrap();

    // Creamos un String mutable (variable algo) vacío
    let mut algo = String::new();

    // Esperamos a que el usuario escriba una palabra
    // y teclee Return.
    // Después colocamos la palabra que haya escrito
    // en la variable 'algo'
    io::stdin().read_line(&mut algo).unwrap();

    // Limpiamos la variable 'algo' de espacios al inicio
    // y al final, y la convertimos en String.
    let algo = algo.trim().to_string();

    // Imprimimos lo que ha escrito el usuario
    println!("✅ Has escrito: {}", algo);
}
```

Comprender en detalle el programa anterior está de momento fuera de nuestro alcance. Pero no es nuestro objetivo averiguar qué hace cada palabrita del anterior programa, sino entender lo que hace cada línea porque en el proyecto final que vamos a desarrollar aparecerán bloques muy parecidos a este que hemos analizado. Lo que nos interesa de verdad del proyecto final es todo lo relacionado con los vectores.

En el listado completo del programa que daremos al final, aparecen algunas funciones con un encabezado similar entre ellas pero no idéntico.

- Veamos la función **mostrar_contactos**

```
fn mostrar_contactos(nombres: &Vec<String>, telefonos: &Vec<String>) {
    println!("\n=== 👤 LISTA DE CONTACTOS DE LA AGENDA ===");
```

```

if nombres.is_empty() {
    println!("⚠ La agenda está completamente vacía.");
    return;
}

// Recorremos las posiciones usando un índice numérico 'i'
// desde 0 hasta el tamaño del vector 'nombres'
for i in 0..nombres.len() {
    // Accedemos de forma segura a cada posición usando [i]
    println!("{}", 👤 Nombre: {} | 📞 Teléfono: {}", i + 1,
nombres[i], telefonos[i]);
}
println!("=====");
}

```

La función **mostrar_contactos** tiene dos parámetros que son ambos prestamos de vectores de String. El primer parámetro se denomina **nombres** y el segundo **telefonos**. Cuando llamemos a la función desde el **main** le pasaremos un préstamo del vector **&agenda_nombres** en la primera posición y un préstamo del vector **&agenda_telefonos** en la segunda posición, ambos vectores definidos al principio del **main** con las siguientes instrucciones:

```

let mut agenda_nombres: Vec<String> = Vec::new();
let mut agenda_telefonos: Vec<String> = Vec::new();

```

La llamada a la función en el main tiene la forma:

```

mostrar_contactos(&agenda_nombres, &agenda_telefonos);

```

Donde observamos lo que acabamos de comentar.

¿Qué hace la función por dentro?

- Vamos a razonar con los nombres de los parámetros, **nombres** y **telefonos** pero ten en cuenta que las funciones operan con los argumentos que se le pasen a los parámetros, es decir, con **agenda_nombres**, **agenda_telefonos**.
- Si el vector de **nombres** está vacío, entonces **nombres.is_empty()** devuelve un valor **true**. El if se cumple y por tanto el programa muestra un mensaje y la función termina con un **return**. El programa seguirá en el main en la instrucción siguiente a la llamada a la función.
- Si el **if** no se cumple, pasamos a la instrucción **for**. Aquí, tengamos en cuenta que **nombres.len()** nos da la cantidad de elementos que tiene el vector **nombres**. Es decir, si **nombres** tiene 5 elementos entonces **nombres.len()** es igual a 5.
- En el bucle **for i in 0..nombres.len()**, la variable **i** empieza por valer 0 y en cada pasada del bucle aumenta en uno su valor hasta valer **nombres.len()** que en el caso anterior

hemos supuesto que vale 5. Entonces la variable `i` va tomando los valores 0, 1, 2, 3, 4 porque el último de los valores, el 5, está excluido del rango. Si quisiéramos que el rango hubiese llegado hasta el 5 tendríamos que haber escrito **`for i in 0..=nombres.len()`**.

Para cada valor de `i`, se ejecutan todas las instrucciones que hay dentro del `for`. En este caso solo hay una.

Para `i=0` se ejecutará:

```
println!("{}", 👤 Nombre: {} | 📞 Teléfono: {}", 0 + 1, nombres[0],
telefonos[0]);
```

Para `i=1` se ejecutará:

```
println!("{}", 👤 Nombre: {} | 📞 Teléfono: {}", 1 + 1, nombres[1],
telefonos[1]);
```

Si hemos supuesto que tenemos 5 valores en el vector, el último `println!` sería:

```
println!("{}", 👤 Nombre: {} | 📞 Teléfono: {}", 4 + 1, nombres[4],
telefonos[4]);
```

El resultado sería un listado con los nombres y telefonos de los contactos.

- Analicemos ahora la función **`añadir_contacto`**

```
fn añadir_contacto(nombres: &mut Vec<String>, telefonos: &mut
Vec<String>) {
    println!("\n--- 🆕 Añadir Nuevo Contacto ---");

    // Pedimos el nombre y lo colocamos en la variable "nombre"
    print!("Introduce el nombre: ");
    io::stdout().flush().unwrap();
    let mut nombre = String::new();
    io::stdin().read_line(&mut nombre).unwrap();
    let nombre = nombre.trim().to_string();

    // Pedimos el teléfono y lo colocamos en la variable "telefono"
    print!("Introduce el teléfono: ");
    io::stdout().flush().unwrap();
    let mut telefono = String::new();
    io::stdin().read_line(&mut telefono).unwrap();
    let telefono = telefono.trim().to_string();

    // Guardamos los datos en sus respectivos vectores
    // Las datos se añaden a los que ya hayan
    nombres.push(nombre);
    telefonos.push(telefono);
}
```

```
println!("✅ ¡Contacto guardado con éxito!");
}
```

De esta forma actualizamos los dos vectores «nombres» y «telefonos» con los nuevos valores que hemos introducido por teclado.

- Analicemos ahora la función **buscar_contacto**

```
fn buscar_contacto(nombres: &Vec<String>, telefonos: &Vec<String>) {

    // Indicamos que vamos a buscar por nombre
    println!("\n--- 🔍 Buscar por Nombre ---");

    // Pedimos el nombre del contacto a buscar
    // y lo guardamos en la variable "busqueda"
    print!("¿A quién estás buscando?: ");
    io::stdout().flush().unwrap();
    let mut busqueda = String::new();
    io::stdin().read_line(&mut busqueda).unwrap();
    let busqueda = busqueda.trim().to_string();

    // Definimos inicialmente la variable encontrado a "false"
    let mut encontrado = false;

    // Buscamos en el vector de nombres
    for i in 0..nombres.len() {
        // Comparamos para cada i el nombres guardado en el vector
        // con el que ha tecleado "busqueda" el usuario
        // pasamos ambos a mayúscula con .to_lowercase() para
        // que no influyan mayúscula y minúsculas.

        // Si lo encontramos imprimimos y salimos con "break"
        if nombres[i].to_lowercase() == busqueda.to_lowercase() {
            println!("🎉 ¡Encontrado! El teléfono de {} es: 📞 {}",
nombres[i], telefonos[i]);
            encontrado = true;
            break; // Salimos del bucle for porque ya lo hemos encontrado
        }
    }

    // Si no lo hemos encontrado imprimimos este mensaje
    if !encontrado {
        println!("❌ Lo siento, '{}' no figura en tu lista de
contactos.", busqueda);
    }
}
```

Nos queda solo analizar la función `main` que es por donde empieza el programa a ejecutarse.

- Analicemos la función **main**

```
fn main() {
    // Creamos las bases de datos (vectores) de la
    // agenda (vacías al iniciar)
    let mut agenda_nombres: Vec<String> = Vec::new();
    let mut agenda_telefonos: Vec<String> = Vec::new();

    println!("=====");
    println!(" 📱 ¡BIENVENIDO A TU AGENDA EN RUST! ");
    println!("=====");

    // Iniciamos un bucle "loop" infinito. Solo terminará cuando el
    // usuario elija salir.
    loop {
        // Si el usuario no pulsa ninguna tecla, el programa
        // imprime en cada pasada del bucle las líneas println! de
        // abajo que constituyen el menú y da la sensación de que
        // nada sucede. Sin embargo, como veremos abajo el programa
        // esta esperando a que el usuario pulse una tecla seguida
        // de un Return

        println!("\n🕒 ¿Qué deseas hacer hoy?");
        println!("1. Ver todos los contactos");
        println!("2. Añadir un contacto");
        println!("3. Buscar un contacto");
        println!("4. 🚪 Salir de la aplicación");
        print!("👉 Selecciona una opción (1-4): ");

        // Hace que se pinte inmediatamente en pantalla lo que teclea
        // el usuario
        io::stdout().flush().unwrap();

        // Leemos la opción del usuario por teclado y
        // la asignamos a la variable "opción"
        let mut opcion = String::new();
        io::stdin().read_line(&mut opcion).unwrap();
        let opcion = opcion.trim();

        // Estructura match (como un if gigante)
        // Selecciona la función que es va a ejecutar
        // Segun la tecla pulsada por el usuario
        // Si pulsa "1" se ejecutará:
        //     mostrar_contactos(&agenda_nombres, &agenda_telefonos)

        // Y así sucesivamente. Las pulsaciones correctas son:
```

```
// 1, 2, 3 y 4 seguidas de Return.

// Cualquier otra pulsación se recoge con el guión bajo _
// y se imprime que no es una opción válida.
match opcion {
    "1" => mostrar_contactos(&agenda_nombres, &agenda_telefonos),
    "2" => añadir_contacto(&mut agenda_nombres, &mut
agenda_telefonos),
    "3" => buscar_contacto(&agenda_nombres, &agenda_telefonos),
    "4" => {
        println!("\n👋 ¡Gracias por usar la agenda en Rust!
Cerrando sistema...");
        break; // Rompe el bucle loop y el programa termina
    }
    _ => println!("⚠️ Opción no válida. Por favor, introduce un
número del 1 al 4."),
}
}
```

Una vez que hemos explicado todos los entresijos de este programa, llega el momento de presentar el listado completo del mismo, para que lo puedas pasar al fichero `main.rs` del proyecto **agenda_contactos** que creamos al principio.

Cuando hayas copiado el listado completo que tienes abajo en el fichero `main.rs`, lanza una terminal integrada de Visual Studio Code en la carpeta del proyecto y ejecuta el programa mediante el comando **cargo run**. Interactúa con el programa para ver que todo funciona. Primero que nada tendrás que elegir la opción 2 para introducir algunos contactos.

Listado completo: Proyecto La Agenda de Contactos por Terminal

```
use std::io::{self, Write}; // Necesario para poder leer lo que el
usuario escribe en el teclado

// --- FUNCIÓN 1: MOSTRAR TODOS LOS CONTACTOS ---
// Recibe préstamos de lectura (&) de los vectores porque solo queremos
mirar los datos
fn mostrar_contactos(nombres: &Vec<String>, telefonos: &Vec<String>) {
    println!("\n== 👤 LISTA DE CONTACTOS DE LA AGENDA ==");

    if nombres.is_empty() {
        println!("⚠️ La agenda está completamente vacía.");
        return;
    }

    // Recorremos las posiciones usando un índice numérico desde 0
    hasta el tamaño del vector
    for i in 0..nombres.len() {
        // Accedemos de forma segura a cada posición usando [i]
```

```

        println!("{}",  Nombre: {} |  Teléfono: {}", i + 1,
nombres[i], telefonos[i]);
    }
    println!("=====");
}

// --- FUNCIÓN 2: AÑADIR UN NUEVO CONTACTO ---
// ¡Ojo! Recibe préstamos MUTABLES (&mut) porque vamos a alterar los
// vectores originales (.push)
fn añadir_contacto(nombres: &mut Vec<String>, telefonos: &mut
Vec<String>) {
    println!("\n---  Añadir Nuevo Contacto ---");

    // Pedimos el nombre
    print!("Introduce el nombre: ");
    io::stdout().flush().unwrap(); // Fuerza a la terminal a mostrar el
    texto en pantalla inmediatamente
    let mut nombre = String::new();
    io::stdin().read_line(&mut nombre).unwrap();
    let nombre = nombre.trim().to_string(); // .trim() elimina el
    "Enter" invisible del teclado

    // Pedimos el teléfono
    print!("Introduce el teléfono: ");
    io::stdout().flush().unwrap();
    let mut telefono = String::new();
    io::stdin().read_line(&mut telefono).unwrap();
    let telefono = telefono.trim().to_string();

    // Guardamos los datos en sus respectivos cajones
    nombres.push(nombre);
    telefonos.push(telefono);

    println!("✅ ¡Contacto guardado con éxito!");
}

// --- FUNCIÓN 3: BUSCAR UN CONTACTO ---
fn buscar_contacto(nombres: &Vec<String>, telefonos: &Vec<String>) {
    println!("\n---  Buscar por Nombre ---");
    print!("¿A quién estás buscando?: ");
    io::stdout().flush().unwrap();
    let mut busqueda = String::new();
    io::stdin().read_line(&mut busqueda).unwrap();
    let busqueda = busqueda.trim().to_string();

    let mut encontrado = false;

    // Buscamos en el vector de nombres
    for i in 0..nombres.len() {
        // Comparamos el nombre guardado con la búsqueda del usuario

```

```
(sin importar mayúsculas)
    if nombres[i].to_lowercase() == busqueda.to_lowercase() {
        println!("🔍 ¡Encontrado! El teléfono de {} es: 📞 {}",
nombres[i], telefonos[i]);
        encontrado = true;
        break; // Salimos del bucle porque ya lo hemos encontrado
    }
}

if !encontrado {
    println!("❌ Lo siento, '{}' no figura en tu lista de
contactos.", busqueda);
}
}

// --- FUNCIÓN PRINCIPAL: EL MENU INTERACTIVO ---
fn main() {
    // Creamos las bases de datos de la agenda (vacías al iniciar)
    let mut agenda_nombres: Vec<String> = Vec::new();
    let mut agenda_telefonos: Vec<String> = Vec::new();

    println!("=====");
    println!(" 📱 ¡BIENVENIDO A TU AGENDA EN RUST! ");
    println!("=====");

    // Iniciamos un bucle loop infinito. Solo terminará cuando el usuario
    elija salir.
    loop {
        println!("\n👤 ¿Qué deseas hacer hoy?");
        println!("1. Ver todos los contactos");
        println!("2. Añadir un contacto");
        println!("3. Buscar un contacto");
        println!("4. 🚪 Salir de la aplicación");
        print!("👉 Selecciona una opción (1-4): ");
        io::stdout().flush().unwrap();

        // Leemos la opción del usuario por teclado
        let mut opcion = String::new();
        io::stdin().read_line(&mut opcion).unwrap();
        let opcion = opcion.trim();

        // Estructura match (como un if gigante) para decidir qué función
        ejecutar
        match opcion {
            "1" => mostrar_contactos(&agenda_nombres, &agenda_telefonos),
            "2" => añadir_contacto(&mut agenda_nombres, &mut
agenda_telefonos),
            "3" => buscar_contacto(&agenda_nombres, &agenda_telefonos),
            "4" => {
```

```

        println!("\n👋 ¡Gracias por usar la agenda en Rust!
Cerrando sistema...");
        break; // Rompe el bucle loop y el programa termina
    }
    _ => println!("⚠️ Opción no válida. Por favor, introduce un
número del 1 al 4."),
}
}

// Instrucción para pruebas
println!("Imprimimos desde última línea: {:?}", agenda_nombres)
}

```

🧠 Conclusiones sobre la forma de trabajar de Rust?

Fíjate en las llamadas a las funciones dentro del **match** en la función **main**:

- Para mostrar o buscar un contacto, enviamos como parámetro a la función correspondiente un préstamo **&agenda_nombres**. Le prestamos la agenda a la función para que **pueda leerla pero no podrá modificarla**.
- Para añadir un contacto, enviamos como parámetro a la función correspondiente un préstamo mutable **&mut agenda_nombres**. Le prestamos la agenda a la función y le **damos permiso para que pueda de escribir** contactos nuevos.
- Como en ambos casos pasamos un préstamo a la función, no la variable original **agenda_nombres**, esta no se consume en las llamadas a las funciones y sigue activa al final del main. Para demostrarlo, introduce algún contacto mediante la opción «2» y luego elige la opción «4» que **sale del bucle loop mediante un break y cae en la función main**, ejecutando la línea **println!(«Imprimimos desde última línea: {:?}», agenda_nombres)** demostrando así que accede a la variable **agenda_nombres** y por tanto no ha sido destruida.
- Si intentáramos añadir un contacto a la agenda utilizando la función correspondiente, mientras otra función la estuviera leyendo en un segundo plano, ¡el compilador de Rust haría saltar las alarmas y daría un error para proteger la memoria!.

Para ejecutar tareas en segundo plano hay que utilizar **threads**, que los veremos bastante más adelante.

8. Ejercicios resueltos

8.1. Enunciados

8.1.1. Ejercicio 1: Expresiones vs. Sentencias

Enunciado: Crea una función llamada **calcular_area_rectangulo** que reciba la base y la altura del rectángulo (ambos de tipo `f64`) y devuelva el área. Aprovecha la característica de Rust de devolver el valor de la última línea como una expresión (sin usar la palabra clave `return` ni punto y coma). Luego, llámala desde el `main`.

8.1.2. Ejercicio 2: El peligro del Movimiento (Move) con Strings

Enunciado: Intenta pasar una variable de tipo `String` a una función que simplemente imprima su contenido. Después de llamar a la función, intenta imprimir la variable de nuevo en el `main`. Observa el error de compilación por la regla del propietario único y arréglalo haciendo que la función reciba una referencia (préstamo) en lugar de tomar el `ownership`.

8.1.3. Ejercicio 3: Préstamos Mutables (&mut)

Enunciado: Escribe una función llamada **duplicar_puntuacion** que reciba una referencia mutable a un número entero (**&mut i32**) e incremente su valor multiplicándolo por dos. Modifica el valor dentro de la función y muestra el resultado final desde el `main`.

8.1.4. Ejercicio 4: Inicialización y Lectura de Arrays

Enunciado: Crea un array de tamaño 5 que contenga las temperaturas de los últimos 5 días (valores `f32`). Escribe un programa que acceda e imprima la primera y la última temperatura de la lista de forma manual, y que muestre el tamaño total del array utilizando uno de sus métodos nativos.

8.1.5. Ejercicio 5: Trabajando con Vectores Dinámicos

Enunciado: Crea un vector vacío de números enteros. Añade dinámicamente los números del 10 al 30 (de 10 en 10) utilizando el método correspondiente. Finalmente, utiliza un bucle simple para recorrer el vector e imprimir cada elemento multiplicado por 100.

8.1.6. Ejercicio 6: Casting de tipos

Enunciado: Considera los vectores de enteros **let num = [10, 20, 30, 40, 50];** y **let den = [1, 2, 3, 4, 5];**. Calcula un vector de tipo `[f32; 5]` cuyo elemento `i` sea el cociente entre el elemento `i` del vector **num** y el elemento `i` del vector **den**.

8.2. Soluciones

8.2.1. Ejercicio 1: Expresiones vs. Sentencias

```
fn calcular_area_rectangulo(base: f64, altura: f64) -> f64 {
    base * altura // Expresión: sin punto y coma para retornar el valor
}

fn main() {
    let b = 5.5;
    let a = 10.0;
    let area = calcular_area_rectangulo(b, a);
    println!("El área del rectángulo es: {}", area);
}
```

Explicación: Este ejercicio demuestra la diferencia fundamental entre una sentencia (que realiza una acción y termina en ;) y una expresión (que evalúa un valor). Al omitir el punto y coma en la última línea **base * altura**, Rust entiende automáticamente que ese es el valor que la función debe retornar a quien la llame.

8.2.2. Ejercicio 2: El peligro del Movimiento (Move) con Strings

Código con error

```
fn imprimir_mensaje(mensaje: String) {
    println!("El mensaje copiado es: {}", mensaje);
}

fn main() {
    // la variable texto es propietaria del dato "Hola, Rustaceo"
    let texto = String::from("Hola, Rustaceo");

    // Pasamos la variable texto a la función
    // La propiedad del dato se mueve a la función
    imprimir_mensaje(texto);

    // ⬇️ ❌ --- error, no compilará, la variable texto ya no existe
    println!("El texto original sigue vivo: {}", texto);
}
```

Código (Solución corregida):

```
fn imprimir_mensaje(mensaje: &String) {
    println!("El mensaje copiado es: {}", mensaje);
}

fn main() {
    let texto = String::from("Hola, Rustaceo");

    // Pasamos una referencia (&) para prestar el dato sin perder la
```

```

propiedad
    imprimir_mensaje(&texto);

    // Como solo lo prestamos antes, todavía podemos usarlo aquí
    println!("El texto original sigue vivo: {}", texto);
}

```

Explicación: Si pasamos texto sin el ampersand (&), la función `imprimir_mensaje` se convierte en la dueña del String, destruyéndolo al terminar su ejecución. Al usar `&String`, realizamos un «préstamo» (borrowing), lo que permite al main retener la propiedad del dato y seguir usándolo después.

8.2.3. Ejercicio 3: Préstamos Mutables (&mut)

```

fn duplicar_puntuacion(puntuacion: &mut i32) {
    // Usamos el asterisco (*) para desreferenciar y modificar el valor
    real
    *puntuacion *= 2;    // *puntuacion = *puntuacion * 2
}

fn main() {
    let mut record = 150;    // La variable original DEBE ser mutable

    duplicar_puntuacion(&mut record);    // Pasamos la referencia
    mutable

    println!("El nuevo récord es: {}", record);
}

```

Explicación: Para poder modificar un dato del main con una función y que la modificación sea visible desde el main, necesitamos pasar a la función una referencia o préstamo mutable (&mut) del dato. Dentro de la función, utilizamos el operador de desreferenciación (*) para acceder directamente al valor guardado (*puntuacion) en la dirección de memoria apuntada por la referencia y poder modificarlo.

Como pasamos un préstamo, después de la llamada a la función el dato “record” no se destruye, podemos acceder a él e imprimirlo.

8.2.4. Ejercicio 4: Inicialización y Lectura de Arrays

```

fn main() {
    // Array con tamaño grabado a fuego (5 elementos)
    let temperaturas: [f32; 5] = [18.5, 19.2, 21.0, 17.8, 16.4];

    let primera = temperaturas[0];

    // [temperaturas.len() nos da la longitud
    let ultima = temperaturas[temperaturas.len() - 1];
}

```

```
println!("Primera temperatura: {}°C", primera);
println!("Última temperatura: {}°C", ultima);
println!("Se han registrado un total de {} días.",
temperaturas.len());
}
```

Explicación: Los arrays en Rust tienen una longitud fija que se conoce en tiempo de compilación. En este ejercicio vemos cómo acceder a sus elementos mediante **índices** (empezando desde 0) y cómo utilizar el método nativo **.len()** para obtener dinámicamente la cantidad de elementos almacenados.

8.2.5. Ejercicio 5: Trabajando con Vectores Dinámicos

```
fn main() {
    // Creamos un vector mutable vacío
    let mut puntuaciones = Vec::new();

    // Añadimos elementos dinámicamente
    // (durante la ejecución del programa)
    puntuaciones.push(10);
    puntuaciones.push(20);
    puntuaciones.push(30);

    // Recorremos el vector con un préstamo del vector en el bucle for
    for valor in &puntuaciones {
        println!("Valor amplificado: {}", valor * 100);
    }

    println!("El vector 'puntuaciones' sigue activo: {:?}",
puntuaciones)
}
```

Explicación: A diferencia de los arrays, los vectores (Vec) pueden crecer o encogerse en tiempo de ejecución. El método **.push()** añade elementos al final de la colección. En el bucle for, usamos **&puntuaciones** para recorrer los elementos mediante préstamos, evitando así que el bucle consuma (destruya) el vector.

Podemos comprobar al final que el vector **puntuaciones** sigue activo y lo podemos imprimir. Utilizamos el marcador **{:?}** porque el vector es un dato compuesto y no podemos imprimirlo con **{}**.

8.2.6. Ejercicio 6: Casting de tipos

```
fn main() {
    let num = [10, 20, 30, 40, 50];
    let den = [1, 2, 3, 4, 5];

    let mut resultados = [0f32; 5];
```

```
for i in 0..5 {  
    resultados[i] = num[i] as f32 / den[i] as f32;  
}  
  
println!("El resumptado es: {:?}", resultados)  
}
```

Explicación: Inicializamos el vector **resultados** con todos sus elementos a cero y de tipo f32. Como los vectores **num** y **den** son de numeros enteros, por defecto i32, tenemos que convertirlos mediante **as f32** al realizar las divisiones para que los cocientes sean también de tipo f32.